Kevin Gómez Valderas 2ºDAM 24-25

Práctica Primeros Módulos Odoo

Preparación

• Necesitamos un directorio donde guardar los addons (configurado en la opción addons_path del odoo.conf). Deberías de tenerlo de cuando se instalaron los módulos de OCA.

Creo una carpeta para módulos ya creados y otra para módulos personalizados.

addons_path = /mnt/extra-addons,/mnt/custom-addons

```
[options]
db_host = db
db_port = 5432
db_user = db_user
db_password = db_passwd
addons_path = /mnt/extra-addons,/mnt/custom-addons
```

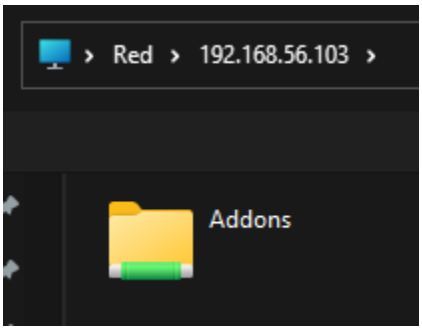
./custom-addons:/mnt/custom-addons

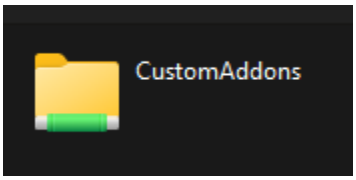
```
version: '3.1'
services:
  web:
    image: odoo:17.0
    container_name: odoo17
    depends_on:
      - db
    ports:
      - "8069:8069"
    volumes:
      - odoo-data:/var/lib/odoo
      - ./config:/etc/odoo
      - ./addons:/mnt/extra-addons
      - ./custom-addons:/mnt/custom-addons
```

- **Compartir el directorio por samba para poder acceder a él desde tu equipo.**

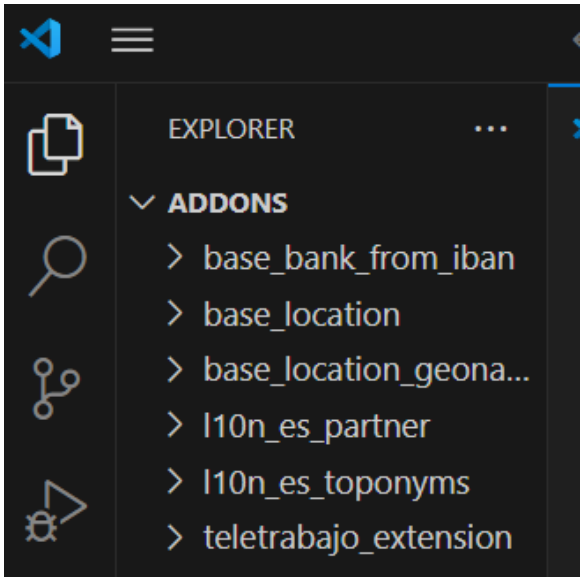
Actualizar Paquetes del Sistema e Instalar samba
sudo apt update sudo apt install samba
Crear carpeta y Conceder Permisos
sudo mkdir /home/kevin/sge/odoo-docker/addons sudo mkdir /home/kevin/sge/odoo-docker/custom-addons sudo chmod -R 777 /home/kevin/sge/odoo-docker/addons sudo chmod -R 777 /home/kevin/sge/odoo-docker/custom-addons
Configurar samba
cd sudo nano /etc/samba/smb.conf
Añadir al final del archivo [Addons] path = /home/kevin/sge/odoo-docker/addons browseable = yes writable = yes valid users = user [CustomAddons] path = /home/kevin/sge/odoo-docker/custom-addons browseable = yes
Crear usuario de samba
sudo smbpasswd -a kevin
Reiniciar Samba
sudo systemctl restart smbd
Comprobación
Acceder desde el equipo local a las carpetas //192.168.56.103

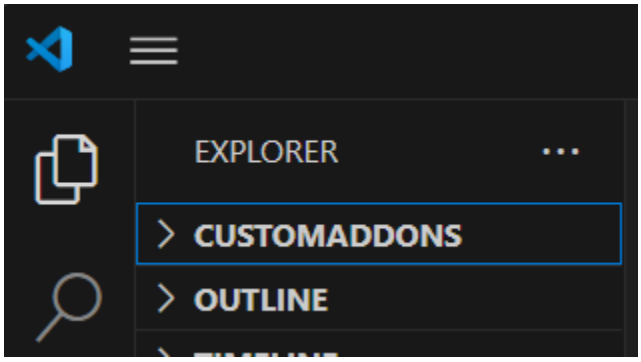
Poner usuario (user) y contraseña (passwd)





• En nuestro equipo tener Visual Studio, Atom o cualquier editor de código que nos guste. Comprobar que puedo acceder al directorio de la máquina virtual (compartido por samba) donde están los addons.

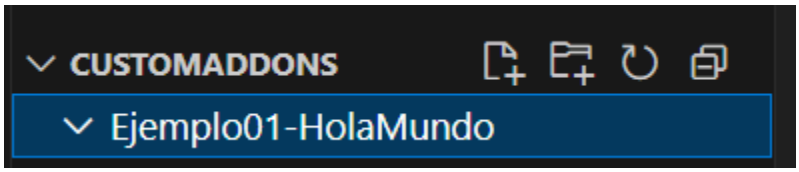




Nuestro primer modulo: “Hola mundo”

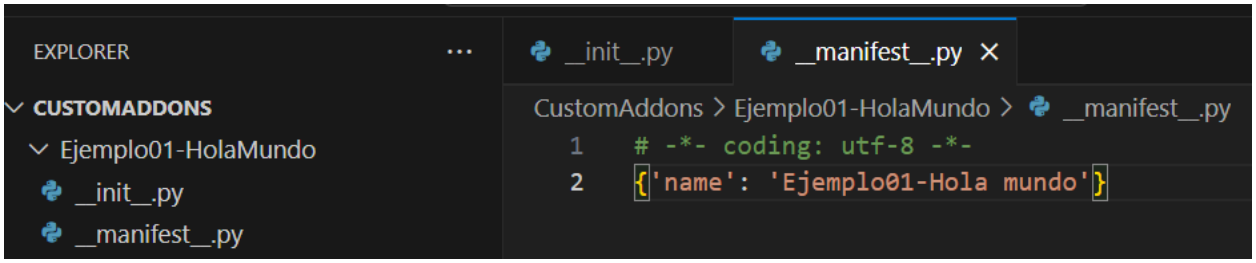
1. Añade el módulo MiMundo_tunombre (di te llamas Serafín, pues mimundo_serafin) siguiendo las siguientes instrucciones.

Para este primer “Hola mundo”, crearemos una carpeta “Ejemplo01-HolaMundo” dentro del directorio que hayamos configurado para colocar nuestros módulos.



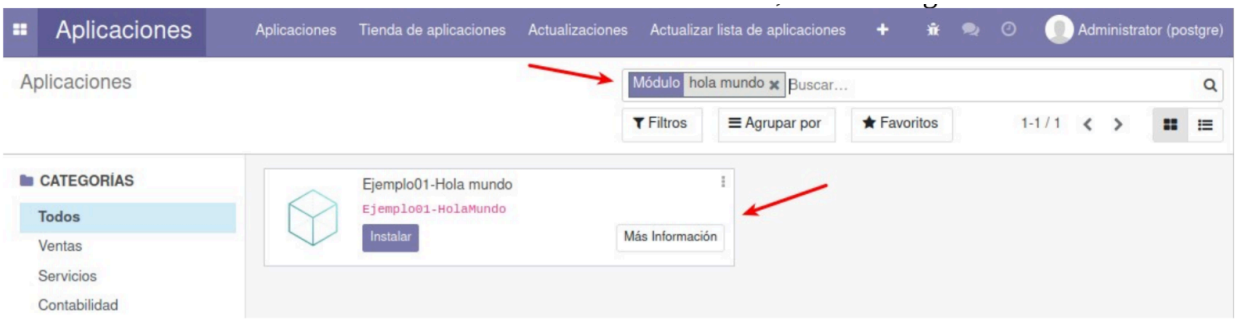
Esta carpeta contendrá dos ficheros:
a) Fichero “__init__.py”: Este fichero estará vacío.
b) Fichero “__manifest__.py”: Este fichero contendrá el siguiente código:

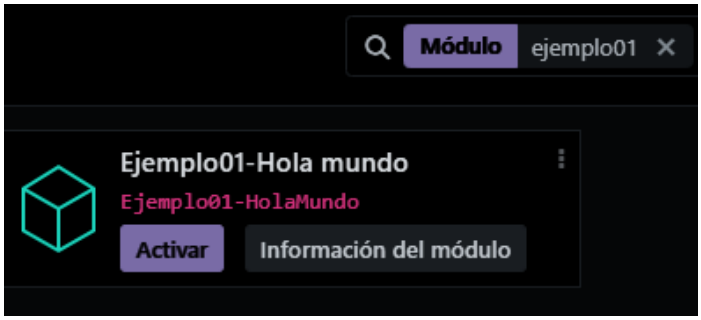
```
# -*- coding: utf-8 -*-
{'name': 'Ejemplo01-Hola mundo'}
```



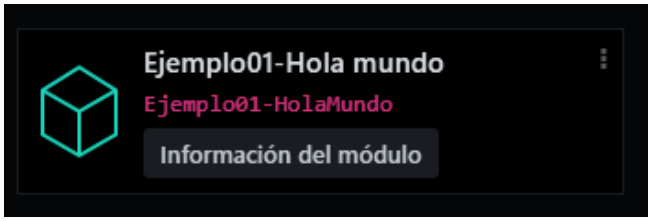
Una vez creada la estructura, con el “Modo Desarrollador” activado, podréis ir al listado de módulos y “Actualizar la lista de aplicaciones” tal como se observa en la imágen

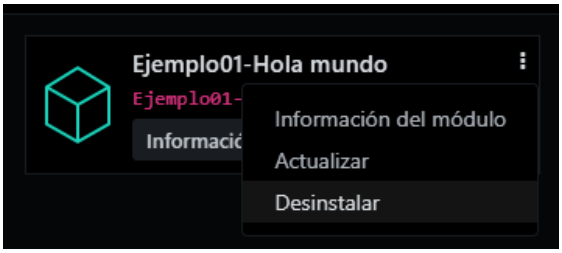
Tras ello, eliminando los filtros de búsqueda por defecto y buscando “hola mundo”, podremos encontrar nuestro módulo. Si todo ha funcionado correctamente, veremos algo similar a:





2. Ahora podremos instalar nuestro módulo para probarlo (aunque este ejemplo no hace nada). Desinstálalo.

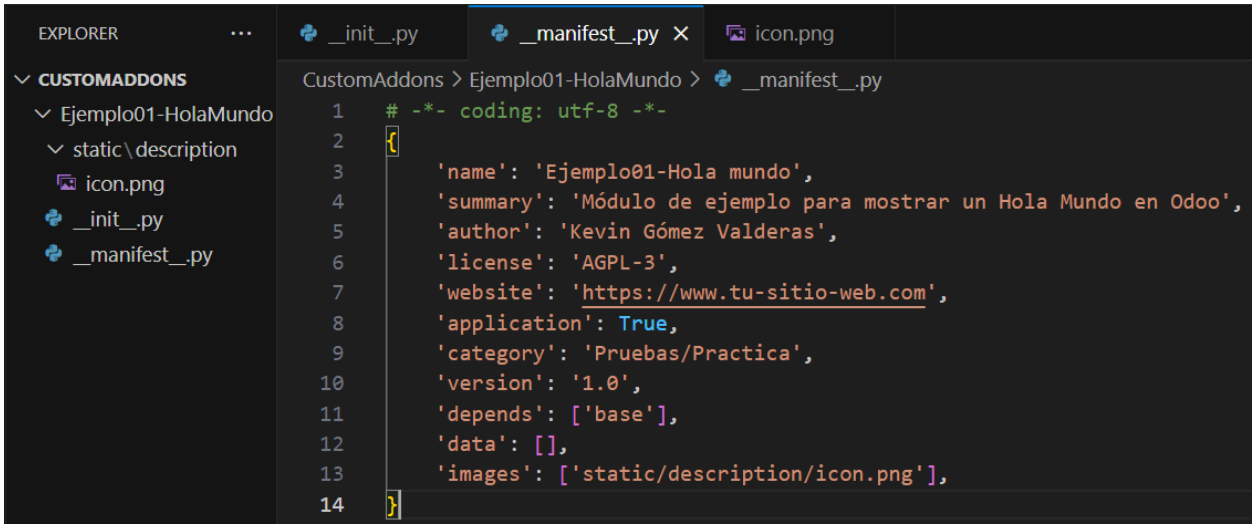




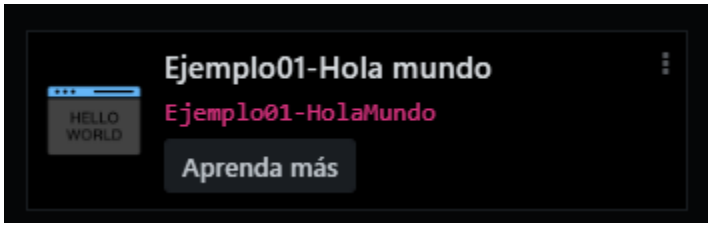
3. En la página 74 del libro Odoo Essentials tienes información de algunos de los campos que puedes rellenar en el fichero __manifest__.py. Añade la siguiente información:

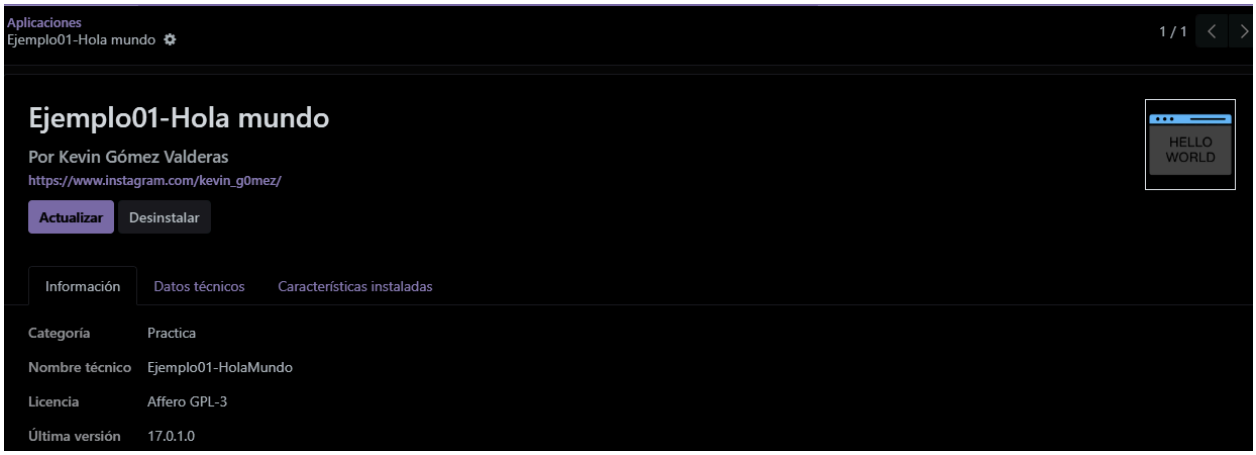
a) Nombre del módulo
b) Resumen
c) Autor
d) Licencia (elige una con sentido)
e) Página web
f) Configura el flag aplicación para que salga el módulo cuando tienes el filtro Apps
g) Añade el módulo a la categoria Pruebas/Practica
h) Añade un icono

```
# -*- coding: utf-8 -*-
{
    'name': 'Ejemplo01-Hola mundo',  # a) Nombre del módulo
    'summary': 'Módulo de ejemplo para mostrar un Hola Mundo en Odoo',  # b) Resumen
    'author': 'Kevin Gómez Valderas',  # c) Autor
    'license': 'AGPL-3',  # d) Licencia (AGPL-3 es una licencia común en Odoo)
    'website': 'https://www.instagram.com/kevin_g0mez/',  # e) Página web
    'application': True,  # f) Configura el flag aplicación para que salga el módulo cuando tienes el filtro Apps
    'category': 'Pruebas/Practica',  # g) Añade el módulo a la categoría Pruebas/Practica
    'version': '1.0',
    'depends': ['base'],
    'data': [
        # Aquí puedes añadir archivos XML, CSV, etc., si los tienes
    ],
    'images': ['static/description/icon.png'],  # h) Añade un icono (asegúrate de tener el archivo icon.png en la
carpeta static/description)
}
```



docker compose restart → Actualizar lista de aplicaciones



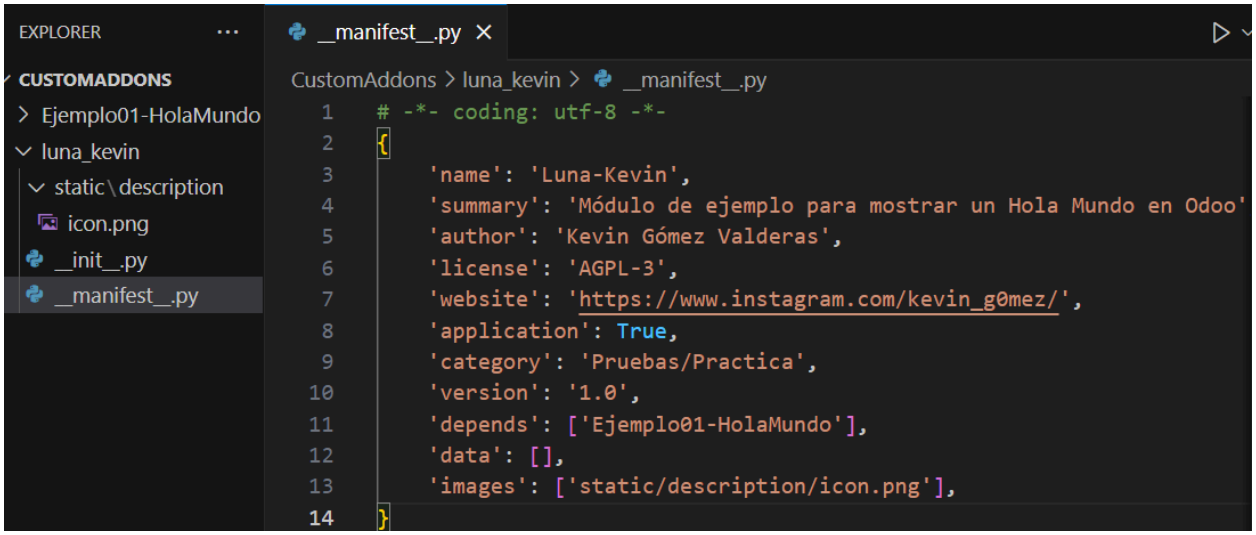


4. Crea un nuevo módulo que se llame luna_tunombre,

a) Configúralo de manera similar al módulo Mundo.
b) En las dependencias configura para que dependa del módulo Mundo.

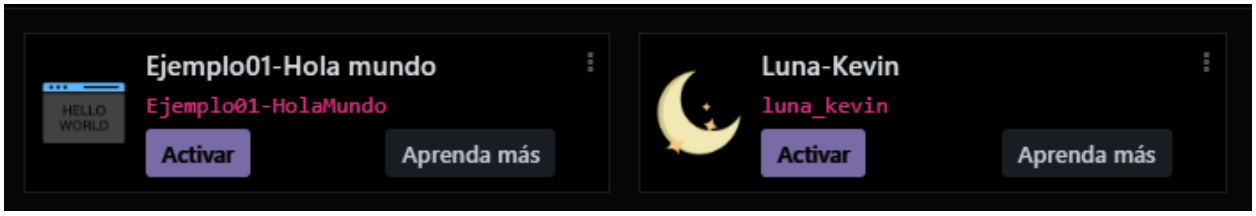
```
# -*- coding: utf-8 -*-
{
    'name': 'Luna-Kevin',
    'summary': 'Módulo de ejemplo para mostrar un Hola Mundo en Odoo',
```

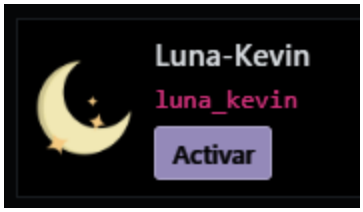
```
    'author': 'Kevin Gómez Valderas',
    'license': 'AGPL-3',
    'website': 'https://www.instagram.com/kevin_g0mez/',
    'application': True,
    'category': 'Pruebas/Practica',
    'version': '1.0',
    'depends': ['Ejemplo01-HolaMundo'],
    'data': [],
    'images': ['static/description/icon.png'],
}
```

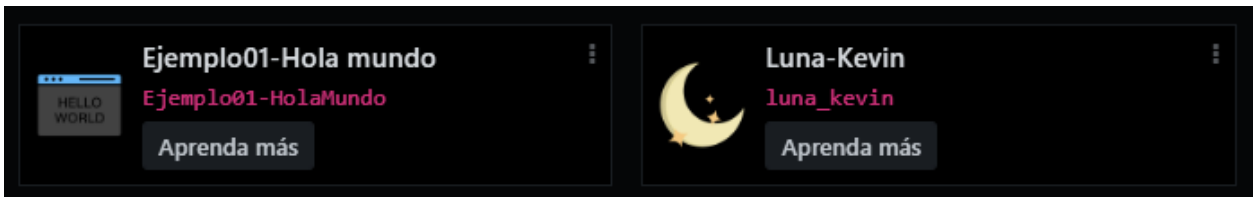


docker compose restart → Actualizar lista de aplicaciones

Con los dos módulos desinstalados instala el módulo Luna. Al depender de Mundo se debe de instalar Mundo automáticamente.




Luna-Kevin
luna_kevin
Activar


HELLO WORLD
Ejemplo01-Hola mundo
Ejemplo01-HolaMundo
Aprenda más
Luna-Kevin
luna_kevin
Aprenda más

Comando scaffold

Los ficheros necesarios que aparecen en un módulo de Odoo siguen siempre la misma estructura

```
library_app/
├── __init__.py
├── __manifest__.py
├── controllers
│   ├── __init__.py
│   └── controllers.py
├── demo
│   └── demo.xml
├── models
│   ├── __init__.py
│   └── models.py
├── security
│   └── ir.model.access.csv
└── views
    ├── templates.xml
    └── views.xml
```

Vamos a realizar una breve explicación de cada uno los ficheros generados:

- "__manifest__.py": es el manifiesto del módulo. Incluye información como el título, descripción, así como ficheros a cargar.
- "views": describe las vistas de nuestro módulo (formulario, árbol ,menús, etc.).
- "demo": incluye datos "demo" para el ejemplo propuesto de modelo.
- "security": Aquí se guardarán los ficheros de seguridad.
- "controllers": aquí tendremos "métodos" para añadir control sobre los datos más allá de la declaración del modelo.

Con el comando scaffold creamos la estructura básica de ficheros para la implementación de un módulo. Por ejemplo, si quiero crear el módulo lista_tareas y el directorio donde tengo los addons es /mnt/extra-addons el comando sería:

```
odoo scaffold lista_tareas /mnt/extra-addons/
```

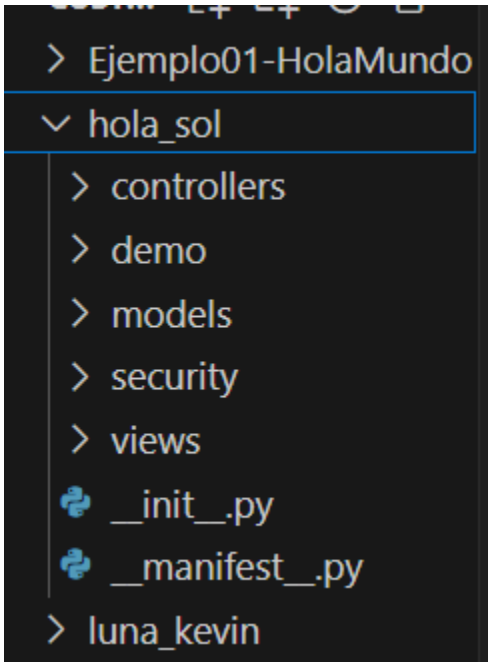
Si estás utilizando docker composer deberás ejecutar el comando utilizando docker exec

docker exec contenedor comando

1. Crea un módulo llamado hola_sol utilizando el comando scaffold. Modifica algunos parámetros del fichero manifest e instálalo.

docker exec --user root -it odoo17 bash
odoo scaffold hola_sol /mnt/custom-addons
chmod -R 777 /mnt/custom-addons/hola_sol

```
kevin@kevinserver:~/sge/odoo-docker$ docker exec --user root -it odoo17 bash
root@1762cedd80d7:/# odoo scaffold hola_sol /mnt/custom-addons
root@1762cedd80d7:/# chmod -R 777 /mnt/custom-addons/hola_sol
root@1762cedd80d7:/#
```




```
# -*- coding: utf-8 -*-
{
    'name': "hola_sol",


    'summary': """
        Módulo de ejemplo para saludar al sol.
        Este módulo demuestra cómo crear un módulo básico en Odoo.
    """,
```

```
    'description': """
        Este módulo es un ejemplo de cómo crear un módulo en Odoo.
        Incluye funcionalidades básicas como vistas, menús y seguridad.
        Puede ser utilizado como base para desarrollar módulos más
complejos.
    """,


    'author': "Kevin Gómez Valderas",
    'website': "https://www.instagram.com/kevin_g0mez/",


    # Categorías disponibles en Odoo. Puedes elegir una o más.
    # Lista completa:
https://github.com/odoo/odoo/blob/15.0/odoo/addons/base/data/ir_module_cat
egory_data.xml
    'category': 'Tools',
    'version': '0.1',


    # Módulos necesarios para que este funcione correctamente
    'depends': ['base', 'web'],


    # Archivos de datos que se cargan siempre
    'data': [
        #'security/ir.model.access.csv',  # Archivo de seguridad para
permisos
        #'views/menu_views.xml',          # Archivo para definir menús
        'views/views.xml',               # Archivo para definir vistas
        'views/templates.xml',           # Archivo para plantillas (por
ejemplo, QWeb)
        #'data/data.xml',                 # Archivo para datos iniciales
    ],


    # Archivos de datos que se cargan solo en modo demostración
    'demo': [
```

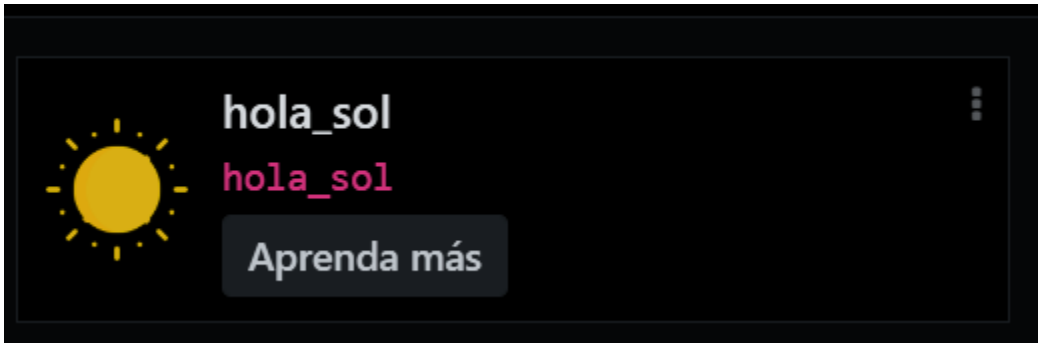
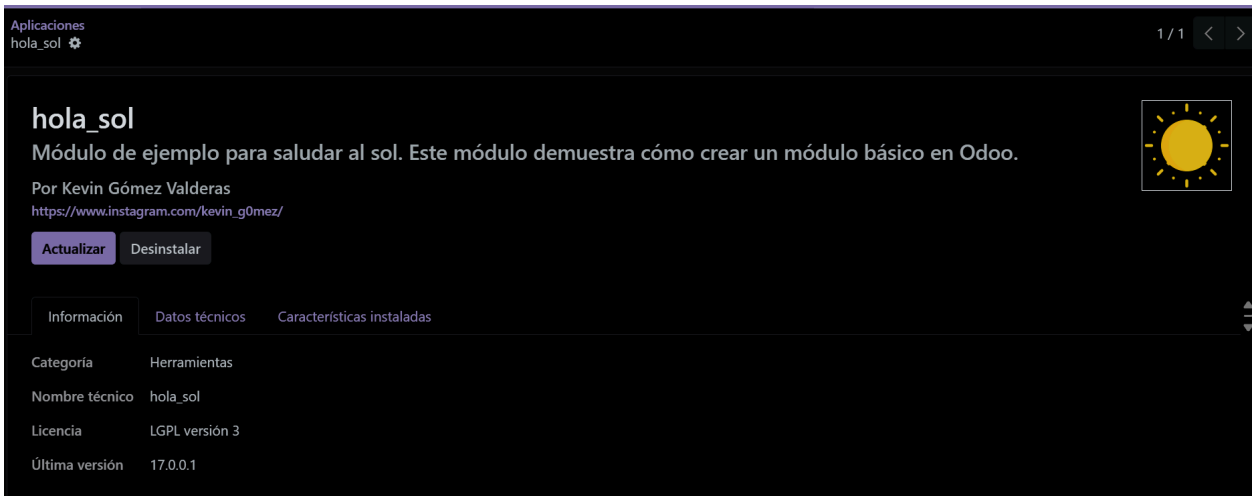
```
        'demo/demo.xml',                  # Archivo de datos de
demostración
    ],


    # Configuración adicional
    'installable': True,                  # Indica si el módulo es
instalable
    'application': True,                  # Indica si el módulo es una
aplicación
    'auto_install': False,                # Indica si el módulo se instala
automáticamente
    'license': 'LGPL-3',                  # Licencia del módulo


    # Precio y moneda del módulo (opcional, para módulos de pago)
    'price': 0.0,
    'currency': 'EUR',


    # Soporte y contacto (opcional)
    'support': 'support@yourcompany.com',
    'maintainer': 'My Company',


    # Imágenes para el módulo en la tienda de aplicaciones (opcional)
    'images': [
        'static/description/icon.png',    # Icono del módulo
        'static/description/banner.png', # Banner del módulo
    ],


    # Dependencias externas (opcional)
    'external_dependencies': {
        'python': [],                     # Lista de dependencias de Python
        'bin': [],                        # Lista de dependencias binarias
    },
}
```

docker compose restart → Actualizar lista de aplicaciones